

COMP3311

Week 4 Tutorial



Agenda

✓ Week 4 notices

✓ Week 3 key concepts

✓ Tutorial questions

Week 4 notices

- Quiz 3 (Due this Friday 11:59pm)
- Help sessions are now available
 - They're running in week 4 and week 5 Mon-Fri (timetable on WebCMS)
- Assignment 1 due 23:59 Friday 3 July (Week 5 Friday).
 - Whenever you submit, please check on a CSE computer first that your SQL code works on a fresh database (spec would also encourage this).
 - Manually check your code yourself, as well as run the autotests

Week 3 Key Concepts

Overview

- SQL **INSERT**
- SQL **UPDATE** Statements
- SQL **DELETE**
- SQL Queries
 - **FROM** and **JOIN**
 - **WHERE**
 - **GROUP BY**
 - **HAVING**
 - **SELECT**
 - **ORDER BY**
 - **LIMIT** and **OFFSET**

Week 3 Key Concepts

SQL Insert Statements

- To insert a row/tuple into a table, use an INSERT INTO statement.

```
INSERT INTO Table_Name VALUES (1, 'data', ...)
```

- Note: The values of the tuple needs to correspond to the order of declaration of the table
- Alternative syntax:

```
INSERT INTO Table_Name(Att1, Att2, ...) VALUES (1, 'data', ...)
```


Week 3 Key Concepts

SQL Update Statements

- To update certain rows of our database, use an UPDATE statement
 - This will update all rows in the table that match the condition

```
UPDATE table_name
SET    att1 = newval1, att2 = newval2...
WHERE condition
```

- e.g.

<u>accNo</u>	branch	balance
1	Sydney	-1000
2	Burwood	28000
3	Sydney	11111

```
UPDATE accounts
SET    balance = 10000
WHERE balance < 0
```

<u>accNo</u>	branch	balance
1	Sydney	10000
2	Burwood	28000
3	Sydney	11111

Week 3 Key Concepts

SQL Delete Statements

```
DELETE FROM TableName WHERE condition;  
DROP TABLE TableName;
```

- Deleting rows or tables can lead to potential problems such as violating **referential integrity**
 - all foreign key values must be referencing an instance that does exist

Week 3 Key Concepts

SQL Delete Statements

- Three main ways to handle this deletion when designing the schema:

1. Prevent Deletion (Default behaviour for SQL)

- Suppose we run: **DELETE FROM Department WHERE id = 1;**

Department

id	name
1	Sales
2	Eng

Employee

id	name	dept_id
101	Alice	1
102	Bob	1
103	Carol	2

Week 3 Key Concepts

SQL Delete Statements

- Three main ways to handle this deletion when designing the schema:
 2. Add **ON DELETE CASCADE** after the foreign key declaration. When something is deleted, then all foreign keys referring to the deleted tuple also get deleted
 - Suppose we run: **DELETE FROM Department WHERE id = 1;**

Department

id	name
2	Eng

Employee

id	name	dept_id
103	Carol	2

Week 3 Key Concepts

SQL Delete Statements

- Three main ways to handle this deletion when designing the schema:
3. Add **ON DELETE SET NULL** after the foreign key declaration to turn the FK values into **NULL**, or **ON DELETE SET DEFAULT** to turn them into a provided **DEFAULT** value.
 - Suppose we run: **DELETE FROM Department WHERE id = 1;** and deleting Sales keeps the employees but nulls their **dept_id**

Department

id	name
2	Eng

Employee

id	name	dept_id
101	Alice	NULL
102	Bob	NULL
103	Carol	2

Week 3 Key Concepts

SQL Queries

- Queries help find **specific instances of data based on certain desired conditions**. Queries pretty much always **return a table**, although some queries can return a single value (and one-cell table).
- We will look at the logical order that defines what a query means
 - Note: the actual execution engine is free to reorder operations for performance

Week 3 Key Concepts

SQL Queries

```
-- 5th step despite written first, chooses columns. Can also rename them. To select all columns, use SELECT *
SELECT column1, column2 AS new_name
-- 1st step, choose table(s) to search through. You can also give the tables aliases as a nickname (usually single letter)
FROM table_name (alias)
    -- Part of FROM, attach another table based on a condition (usually a PK to FK match)
    JOIN table_name2 (alias) ON (condition)
-- 2nd step, filters out rows, only get rows that fulfill the condition (or conditions)
WHERE condition
-- 3rd step, makes groups out of the remaining rows based on shared values.
GROUP BY grouping_attributes
-- 4th step, acts as a second WHERE on the specific groups.
-- Can also use aggregates like COUNT(att) in the conditions because there are groups
HAVING group_conditions
-- 6th step (often last), order the rows by attribute values. Can only use ones in the SELECT statement.
ORDER BY ordering_attributes (ASC/DESC)
-- Last major step, limit the number of rows, OFFSET skips rows
LIMIT number_rows OFFSET num_skipped_rows
```

- Note: **SELECT** is the only necessary keyword but commonly you would use **FROM** and **WHERE** as well

Week 3 Key Concepts

FROM and JOIN

- **FROM** is the first step, chooses what table/tables to extract data from.
- **JOIN** joins two tables based on a certain condition. Usually that condition is (t1.pk = t2.fk), where t2's foreign key references t1's primary key.

Accounts

<u>accNo</u>	branch	balance
1	Sydney	10000
2	Burwood	28000
3	Sydney	11111

Owns

account	customer
2	202
2	201
3	201

```
SELECT * -- all columns
FROM accounts a
JOIN owns o ON (a.accNo = o.account)
```

accNo	branch	balance	account	customer
2	Burwood	28000	2	202
2	Burwood	28000	2	201
3	Sydney	11111	3	201

Week 3 Key Concepts

Joins

- There are multiple **JOIN** types.
 - **(INNER) JOIN**: Most basic join. Only include rows that can join on the condition
 - **LEFT (OUTER) JOIN**: Joins, but every row in the left/first table will be included even if there's no rows in the right table to join on. Right table's values will be **NULL** in that case.
 - **RIGHT (OUTER) JOIN**: Similar to **LEFT JOIN**, but instead, every row of the right table is included. Not as commonly used as **LEFT**.
 - **FULL (OUTER) JOIN**: Combination of left and right join.

<u>accNo</u>	branch	balance	account	customer
1	Sydney	10000	2	202
2	Burwood	28000	2	201
3	Sydney	11111	3	201

```
SELECT * -- all columns
FROM accounts a
LEFT JOIN owns o ON (accNo = account)
```

accNo	branch	balance	account	customer
1	Sydney	10000	NULL	NULL
2	Burwood	28000	2	202
2	Burwood	28000	2	201
3	Sydney	11111	3	201

Week 3 Key Concepts

WHERE

- **WHERE** will filter out rows based on a condition (or conditions). The condition is run **per row**. A lot of these conditions also can be used for **CHECK** constraints.
- Some operations for conditions include:
 - *val* **LIKE** '*pattern*': Compares string to a pattern. Search yourself for more detail on how this works. (%) matches any sequence of zero or more characters, _ matches exactly one character) (standard SQL)
 - *val* ~ '*regex*': Regex comparison (PostgreSQL specific)
 - <> or != for not equals. >, <, >=, <= for other common operators.
 - *val* **IS NULL** or *val* **IS NOT NULL**
 - *str1* || *str2* for string concatenation, ::*type* for type casting
 - **EXISTS** (*Query*): Checks if query returns any rows. If yes, return true, else return false. Usually subquery contains information about the row to influence the result of **EXISTS**.
 - *col* **IN** (*Query/Tuple of values*): Checks if the *col* value is inside the query or tuple of values. e.g. *col* **IN** ('a', 'b', 'c')
- There are others, but these are just some common ones.
- Use **OR** and **AND** to join conditions together, and **NOT** to negate conditions.

Week 3 Key Concepts

WHERE

- e.g. simple example

Accounts	<u>accNo</u>	branch	balance
	1	Sydney	10000
	2	Burwood	28000
	3	Sydney	11111

```
SELECT * -- all columns
FROM accounts a
WHERE a.branch = 'Sydney'
AND a.balance > 10000
```

Result	<u>accNo</u>	branch	balance
	3	Sydney	11111

Week 3 Key Concepts

GROUP BY, HAVING and Aggregates

- Common aggregates include:

```
COUNT(cols), SUM(col) MAX(col), MIN(col)
```

- e.g. To count number of rows with name Kenneth

```
SELECT COUNT(*) FROM x WHERE x.name = 'Kenneth';
```

- **GROUP BY** will group the remaining rows/tuples into groups, based on sharing common values between attributes. Then aggregates in the **SELECT** statement will apply to each group instead. Think of squishing rows together to make groups (e.g. count per name)
- **HAVING** acts as a second **WHERE** (but now with access to aggregates) to get rid of certain groups.

Week 3 Key Concepts

SELECT, ORDER BY and LIMIT

- **SELECT** grabs certain columns from the table produced by the previous steps
 - You can use **DISTINCT** to get distinct values. **DISTINCT** will apply to all columns, **DISTINCT ON** (atts) can specify certain attributes
 - You can also run functions on columns

```
e.g. LOWER(str), col1 || col2
```

- **ORDER BY** allows you to use: **ASC** / **DESC** — sort direction. **ASC** is ascending (the default, so usually omitted) and **DESC** is descending
- **LIMIT** causes the result to be limited to a number of rows you specify
- You can also set an **OFFSET** to skip some initial rows
 - e.g. this ignores the first row, and returns a table with the next two rows.

```
LIMIT 2 OFFSET 1
```

Week 3 Key Concepts

Set Operations and Misc

- You can also combine similar tables/queries in multiple ways:

- `(Query1) UNION (Query2)`: All rows/tuples in *Query1* **OR** *Query2*
- `(Query1) INTERSECT (Query2)`: All rows/tuples in *Query1* **AND** *Query2*
- `(Query1) EXCEPT (Query2)`: All rows/tuples in *Query1* **NOT IN** *Query2*

- These work row-wise whereas **JOINS** work column-wise
- You can also use subqueries, e.g. if a query returns a single value, you can use it in brackets in other queries.

```
SELECT accNo, balance
FROM accounts
WHERE balance > (SELECT AVG(balance) FROM accounts);
```

```
SELECT accNo, branch
FROM accounts
WHERE accNo IN (SELECT account FROM owns);
```

```
SELECT branch, total
FROM (SELECT branch, SUM(balance) AS total
      FROM accounts
      GROUP BY branch) AS sums
WHERE total > 20000;
```

Week 3 Key Concepts

Views

- A view is a stored table that is created from a query, that the user can VIEW whenever. They change when the database changes.
 - Views are helpful to make queries or other views simpler.

```
CREATE (OR REPLACE) VIEW View_Name AS (Query);  
CREATE (OR REPLACE) VIEW View_Name (Attribute_Names) AS (Query);  
DROP VIEW View_Name;
```

Tutorial Questions

Now let's try some more questions :D

- Q2
- Q3
- Q5
- Q8
- Q12
- Q13
- Q15
- Q16
- Q17
- Q19
- Q21
- Q22